

Information Retrieval (Google search)

Eric Atwell and Stuart Roberts,
School of Computing, University of Leeds

IR v Database SQL

Inverted file for efficient match

Boolean set theoretic model v weighted vector model

Worked examples

Evaluation

Query broadening to improve matching

The problem of finding strings in a database

- the simple approach to searching for a keyword uses leading (and trailing) wildcards: eg '%graphics%'
- 'brute force scan' to match such a condition with the text data records held in a traditional relational database.

The relational problem

- Rather than hold full text, why not do content analysis, extract the index terms (keywords), and hold these in a relational database?



module(module_code, title, semester, ...)

term(term_id, value)

index(module_code, term_id)

Sample SQL query: OR

find all modules matching:

'database' or 'AI' or 'knowledge base'

```
select distinct m.* from  
module m inner join index i on m.code = i.mod_code inner  
join term t on t.term_id = i.term_id  
where  
t.value = 'database' OR t.value = 'AI' OR t.value =  
'knowledge base' ;
```

Another sample query: AND (?)

find all modules matching:

'database' and 'AI' and 'knowledge base'

```
select distinct m.* from
module m inner join index i on m.code = i.mod_code inner
join term t on t.term_id = i.term_id
where
t.value = 'database' AND t.value = 'AI' AND t.value =
'knowledge base' ;
```

This SQL query will not match any record; t.value cannot be simultaneously equal to 'database', 'AI' and 'knowledge base'.

We cannot simply replace the 'OR's of the last SQL query with 'AND's.

Corrected sample query: AND

find all modules matching: 'database' and 'AI'

```
select distinct m.* from
module m inner join index i1 on m.code = i1.mod_code inner
join term t1 on t1.term_id = i1.term_id
inner join index i2 on m.code = i2.mod_code inner join term
t2 on t2.term_id = i2.term_id
where
t1.value = 'database' and t2.value = 'AI';
```

Both tables 'index' and 'term' must be searched twice in order to establish whether, for each module, it is attached to both terms 'database' and 'AI'.

If the query is a conjunction of N terms, the SQL would have 2N inner joins. AND is more complicated than OR (but common in IR)

Inverted file

- Non-DB structure, so not suitable for standard SQL
- each index term entry 'points' to a list of document record identifiers (RIDs)
- standard indexing method for IR systems
- widely used for search engines
- can be extended to allow for positional (context) searches

Inverted file structure

The idea of an inverted file is, as well as storing a document with its list of terms that are used to index it, we store the list of terms used in the whole collection of documents, and for each term point to the list of documents that are indexed by the term. So we have 'inverted' the structure:

$D_1: T_{11}, T_{12}, \dots, T_{1k}$

$D_2: T_{21}, T_{22}, \dots, T_{2l}$

...

to give:

$T_1: D_{11}, D_{12}, \dots, D_{1m}$

$T_2: D_{21}, D_{22}, \dots, D_{2n}$

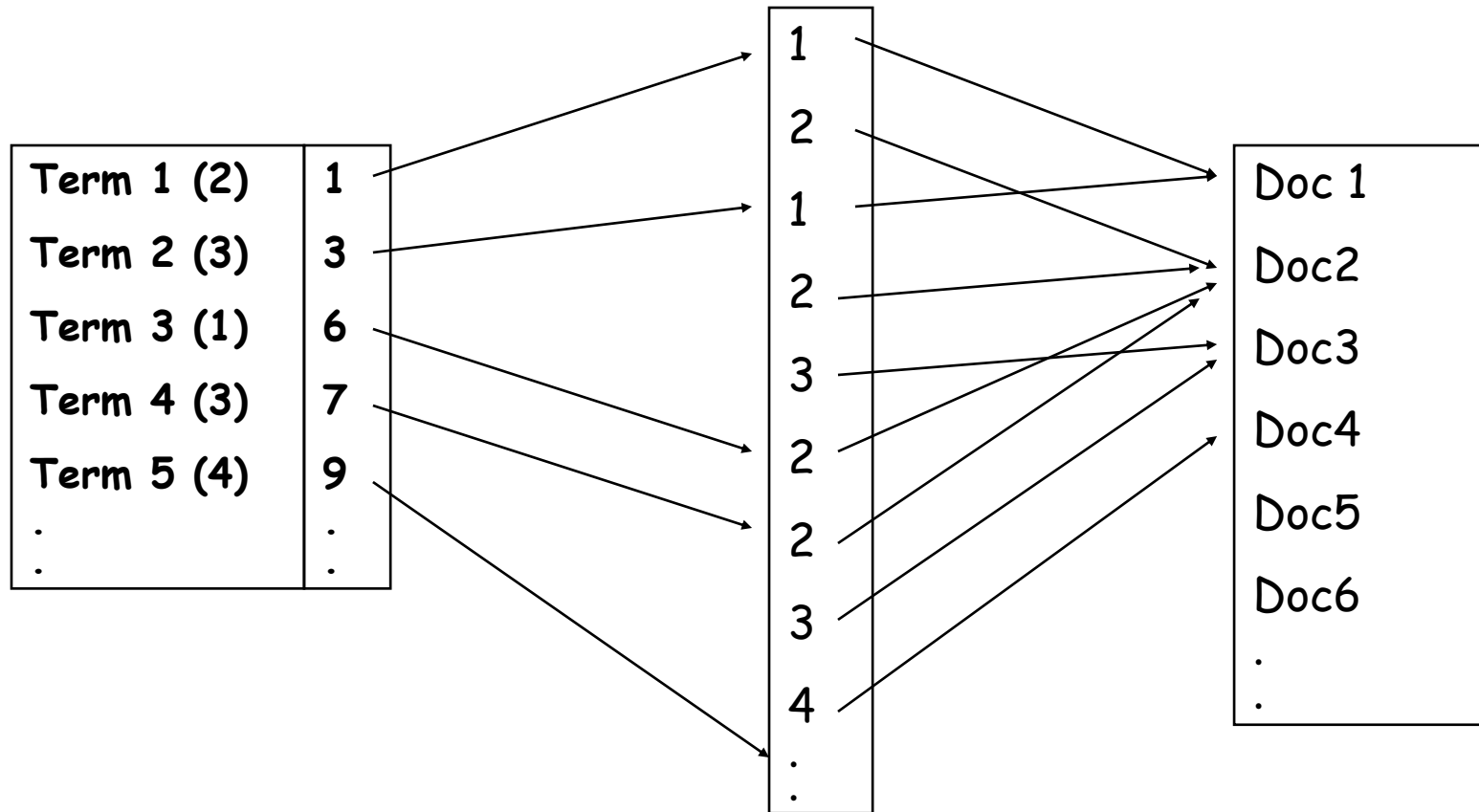
...

Inverted file structure

dictionary

Inverted or postings file

Data file

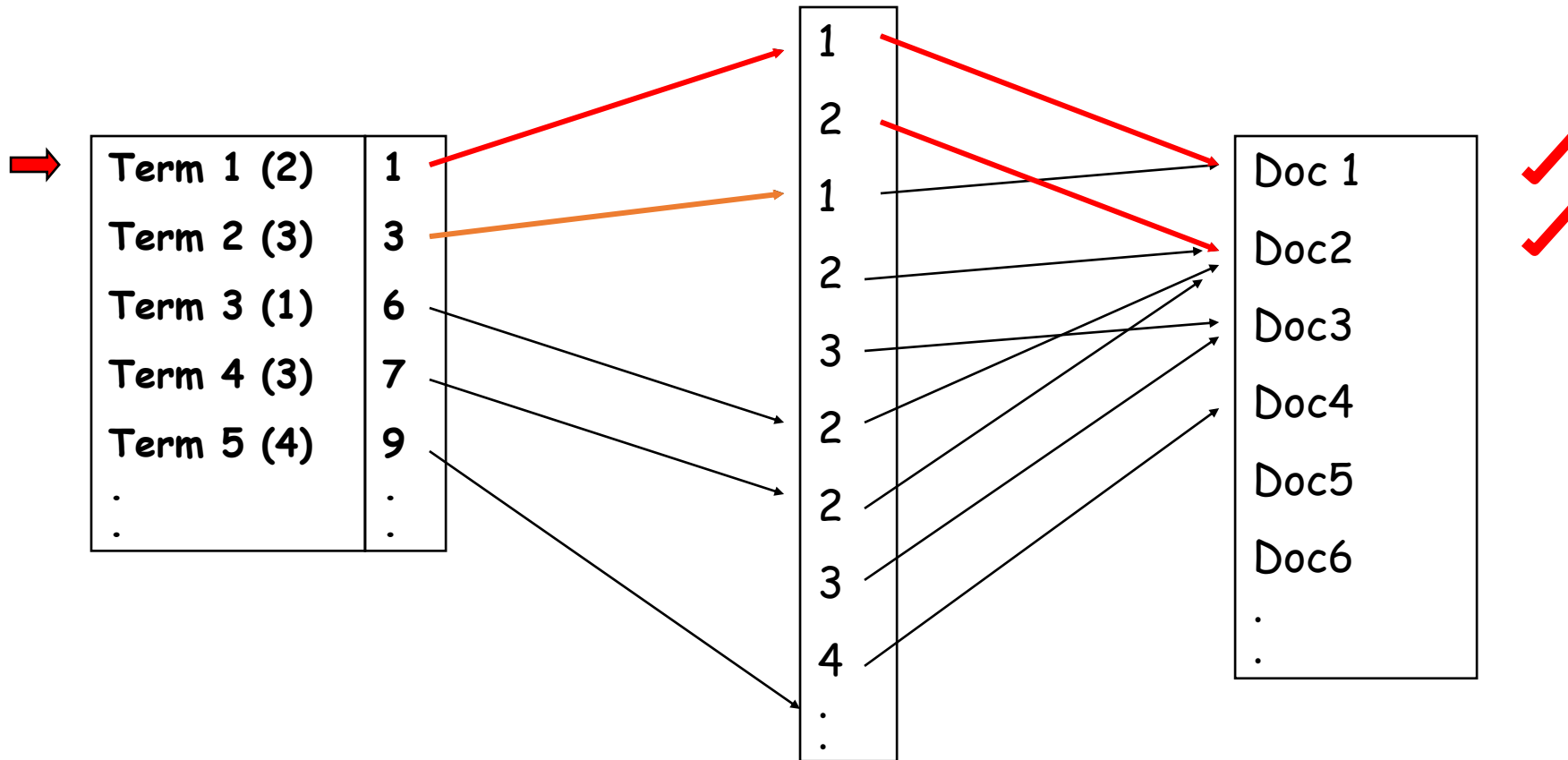


Inverted file structure

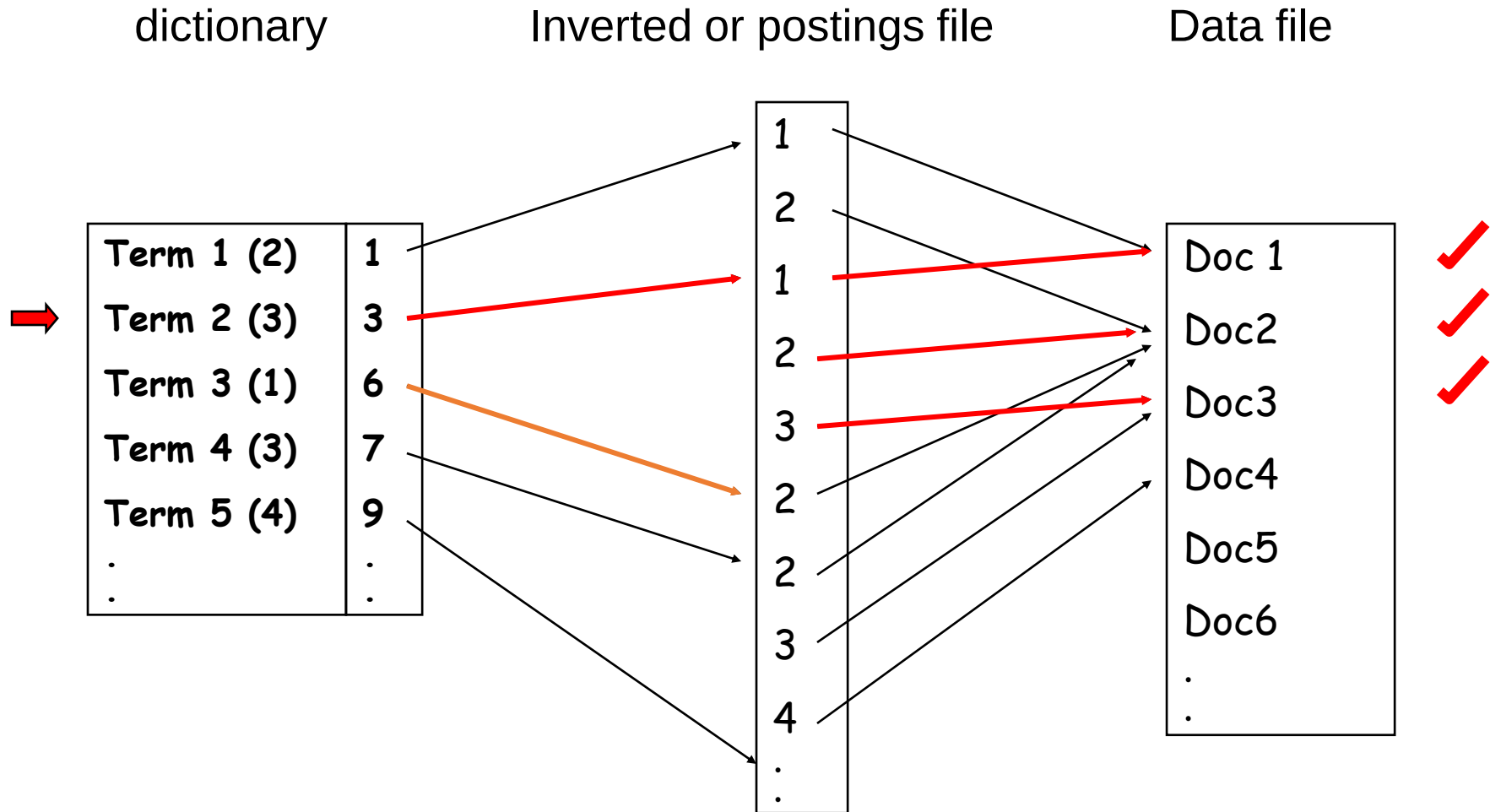
dictionary

Inverted or postings file

Data file



Inverted file structure

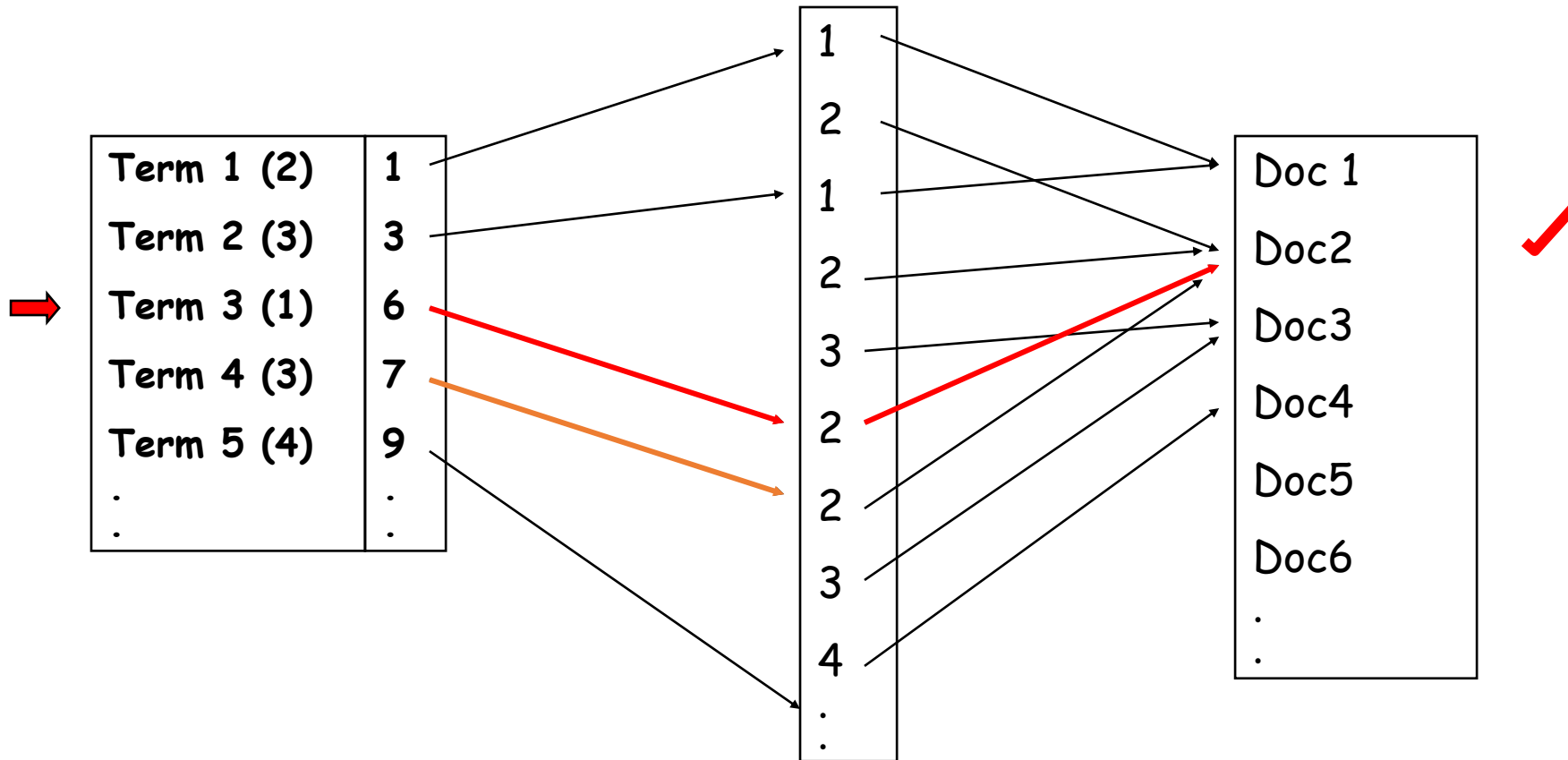


Inverted file structure

dictionary

Inverted or postings file

Data file



Dictionary (in IR)

- list of terms including 'normalised' keywords or stems plus object descriptors (eg author name)
- frequency with which that term occurs in the collection
- pointer to the inverted file
- access to dictionary is by standard file access method (eg binary search or Btree or hashing algorithm)

Inverted file

- for each entry in the dictionary:
 - a list of pointers into the data file (or object-ids, or URLs..)
 - identifying those objects indexed by the dictionary term
- inverted file may also contain:
 - positional information within each document – where?
 - term frequency (or weight) within each document – how many?

Use of inverted file

- Boolean query: $(A \text{ or } B) \text{ and } C$
 - disjunctive normal form: $(A \text{ and } C) \text{ or } (B \text{ and } C) \text{ or } (A \text{ and } B \text{ and } C)$
 $(1, 0, 1) \text{ OR } (0, 1, 1) \text{ OR } (1, 1, 1)$
 - retrieve lists of document ids from inverted file corresponding to A, B and C

doc1	doc2	doc1
doc3	doc3	doc2
doc4	doc5	doc4
doc7	doc6	doc9
doc8	doc8	doc11
doc10	doc12	doc12

Use of inverted file

- Boolean query: $(A \text{ or } B) \text{ and } C$
 - disjunctive normal form:
 $(1, 0, 1) \text{ OR } (0, 1, 1) \text{ OR } (1, 1, 1)$
 - retrieve lists of document ids from inverted file corresponding to A, B and C

doc1

doc3

doc4

doc7

doc8

doc10

doc2

doc3

doc5

doc6

doc8

doc12

doc1

doc2

doc4

doc9

doc11

doc12

doc1: (1, 0, 1)

Use of inverted file

- Boolean query: $(A \text{ or } B) \text{ and } C$
 - disjunctive normal form:
 $(1, 0, 1) \text{ OR } (0, 1, 1) \text{ OR } (1, 1, 1)$
 - retrieve lists of document ids from inverted file corresponding to A, B and C

doc3
doc4
doc7
doc8
doc10

doc2
doc3
doc5
doc6
doc8
doc12

doc2
doc4
doc9
doc11
doc12

doc1: (1, 0, 1)
doc2: (0, 1, 1)

Use of inverted file

- Boolean query: $(A \text{ or } B) \text{ and } C$
 - disjunctive normal form:
 $(1, 0, 1) \text{ OR } (0, 1, 1) \text{ OR } (1, 1, 1)$
 - retrieve lists of document ids from inverted file corresponding to A, B and C

doc3

doc4

doc7

doc8

doc10

doc3

doc5

doc6

doc8

doc12

doc4

doc9

doc11

doc12

doc1: (1, 0, 1)

doc2: (0, 1, 1)

doc3: (1, 1, 0)

Use of inverted file

- Boolean query: $(A \text{ or } B) \text{ and } C$
 - disjunctive normal form:
 $(1, 0, 1) \text{ OR } (0, 1, 1) \text{ OR } (1, 1, 1)$
 - retrieve lists of document ids from inverted file corresponding to A, B and C

doc4

doc7

doc8

doc10

doc5

doc6

doc8

doc12

doc4

doc9

doc11

doc12

doc1: (1, 0, 1)

doc2: (0, 1, 1)

doc3: (1, 1, 0)

doc4: (1, 0, 1)

Use of inverted file

- Boolean query: $(A \text{ or } B) \text{ and } C$
 - disjunctive normal form:
 $(1, 0, 1) \text{ OR } (0, 1, 1) \text{ OR } (1, 1, 1)$
 - retrieve lists of document ids from inverted file corresponding to A, B and C

doc7
doc8
doc10

doc5
doc6
doc8
doc12

doc9
doc11
doc12

doc1: (1, 0, 1)
doc2: (0, 1, 1)
doc3: (1, 1, 0)
doc4: (1, 0, 1)
doc5: (0, 1, 0)

Use of inverted file

- Boolean query: $(A \text{ or } B) \text{ and } C$
 - disjunctive normal form:
 $(1, 0, 1) \text{ OR } (0, 1, 1) \text{ OR } (1, 1, 1)$
 - retrieve lists of document ids from inverted file corresponding to A, B and C

doc7
doc8
doc10

doc6
doc8
doc12

doc9
doc11
doc12

doc1: (1, 0, 1)
doc2: (0, 1, 1)
doc3: (1, 1, 0)
doc4: (1, 0, 1)
doc5: (0, 1, 0)
doc6: (0, 1, 0)

Use of inverted file

- Boolean query: $(A \text{ or } B) \text{ and } C$
 - disjunctive normal form:
 $(1, 0, 1) \text{ OR } (0, 1, 1) \text{ OR } (1, 1, 1)$
 - retrieve lists of document ids from inverted file corresponding to A, B and C

doc7
doc8
doc10

doc8
doc12

doc9
doc11
doc12

doc1: (1, 0, 1)
doc2: (0, 1, 1)
doc3: (1, 1, 0)
doc4: (1, 0, 1)
doc5: (0, 1, 0)
doc6: (0, 1, 0)
doc7: (1, 0, 0)

Use of inverted file

- Boolean query: $(A \text{ or } B) \text{ and } C$
 - disjunctive normal form:
 $(1, 0, 1) \text{ OR } (0, 1, 1) \text{ OR } (1, 1, 1)$
 - retrieve lists of document ids from inverted file corresponding to A, B and C

doc8
doc10

doc8
doc12

doc9
doc11
doc12

doc1: (1, 0, 1)
doc2: (0, 1, 1)
doc3: (1, 1, 0)
doc4: (1, 0, 1)
doc5: (0, 1, 0)
doc6: (0, 1, 0)
doc7: (1, 0, 0)
doc8: (1, 1, 0)

Use of inverted file

- Boolean query: $(A \text{ or } B) \text{ and } C$
 - disjunctive normal form:
 $(1, 0, 1) \text{ OR } (0, 1, 1) \text{ OR } (1, 1, 1)$
 - retrieve lists of document ids from inverted file corresponding to A, B and C

doc10

doc12

doc9

doc11

doc12

doc1: (1, 0, 1)

doc2: (0, 1, 1)

doc3: (1, 1, 0)

doc4: (1, 0, 1)

doc5: (0, 1, 0)

doc6: (0, 1, 0)

doc7: (1, 0, 0)

doc8: (1, 1, 0)

doc9: (0, 0, 1)

Use of inverted file

- Boolean query: $(A \text{ or } B) \text{ and } C$
 - disjunctive normal form:
 $(1, 0, 1) \text{ OR } (0, 1, 1) \text{ OR } (1, 1, 1)$
 - retrieve lists of document ids from inverted file corresponding to A, B and C

doc10

doc12

doc11
doc12

doc1: (1, 0, 1)
doc2: (0, 1, 1)
doc3: (1, 1, 0)
doc4: (1, 0, 1)
doc5: (0, 1, 0)
doc6: (0, 1, 0)
doc7: (1, 0, 0)
doc8: (1, 1, 0)
doc9: (0, 0, 1)
doc10: (1, 0, 0)

Use of inverted file

- Boolean query: $(A \text{ or } B) \text{ and } C$
 - disjunctive normal form:
 $(1, 0, 1) \text{ OR } (0, 1, 1) \text{ OR } (1, 1, 1)$
 - retrieve lists of document ids from inverted file corresponding to A, B and C

doc12

doc11
doc12

doc1: (1, 0, 1)
doc2: (0, 1, 1)
doc3: (1, 1, 0)
doc4: (1, 0, 1)
doc5: (0, 1, 0)
doc6: (0, 1, 0)
doc7: (1, 0, 0)
doc8: (1, 1, 0)
doc9: (0, 0, 1)
doc10: (1, 0, 0)
doc11: (0, 0, 1)

Use of inverted file

- Boolean query: $(A \text{ or } B) \text{ and } C$
 - disjunctive normal form:
 $(1, 0, 1) \text{ OR } (0, 1, 1) \text{ OR } (1, 1, 1)$
 - retrieve lists of document ids from inverted file corresponding to A, B and C

doc12

doc12

doc1: (1, 0, 1)

doc2: (0, 1, 1)

doc3: (1, 1, 0)

doc4: (1, 0, 1)

doc5: (0, 1, 0)

doc6: (0, 1, 0)

doc7: (1, 0, 0)

doc8: (1, 1, 0)

doc9: (0, 0, 1)

doc10: (1, 0, 0)

doc11: (0, 0, 1)

doc12: (0, 1, 1)

Use of inverted file

- Boolean query: **(A or B) and C**
 - disjunctive normal form:
 $(1, 0, 1) \text{ OR } (0, 1, 1) \text{ OR } (1, 1, 1)$

doc1: (1, 0, 1)
doc2: (0, 1, 1)
doc3: (1, 1, 0)
doc4: (1, 0, 1)
doc5: (0, 1, 0)
doc6: (0, 1, 0)
doc7: (1, 0, 0)
doc8: (1, 1, 0)
doc9: (0, 0, 1)
doc10: (1, 0, 0)
doc11: (0, 0, 1)
doc12: (0, 1, 1)

Use of inverted file

- Boolean query: **(A or B) and C**
 - disjunctive normal form:
 $(1, 0, 1) \text{ OR } (0, 1, 1) \text{ OR } (1, 1, 1)$

doc1: (1, 0, 1)
doc2: (0, 1, 1)
doc4: (1, 0, 1)
doc12: (0, 1, 1)

report number of hits to user (4)

(Note: can be done before any 'hits' are retrieved)

retrieve all objects using 'pointers':

doc1, doc2, doc4 and doc12

Use of inverted file with weighted terms

weighted query: $A_{0.5}, B_{0.7}, C_{1.0}$

form weighted vector:

(0.5, 0.7, 1.0)

retrieve lists of document ids from inverted file corresponding to A, B and C with weights

doc1 (.2)	doc2 (.6)	doc1 (.4)
doc3 (.6)	doc3 (.8)	doc2 (.4)
doc4 (.7)	doc5 (.9)	doc4 (.7)
doc7 (.3)	doc6 (.3)	doc9 (.6)
doc8 (.5)	doc8 (.5)	doc11 (.3)
doc10 (.5)	doc12 (.2)	doc12 (.6)

Assumes $\text{sim}(\text{Query}, \text{Document})$ function comparing 2 vectors

Use of inverted file

weighted query: $A_{0.5}, B_{0.7}, C_{1.0}$

form weighted vector:

(0.5, 0.7, 1.0)

retrieve lists of document ids from inverted file corresponding to A, B and C with weights

doc1 (.2)	doc2 (.6)	doc1 (.4)
doc3 (.6)	doc3 (.8)	doc2 (.4)
doc4 (.7)	doc5 (.9)	doc4 (.7)
doc7 (.3)	doc6 (.3)	doc9 (.6)
doc8 (.5)	doc8 (.5)	doc11 (.3)
doc10 (.5)	doc12 (.2)	doc12 (.6)

doc1: 0.85

$\text{sim}((0.5, 0.7, 1.0), (0.2, 0.0, 0.4)) = 0.85$

Use of inverted file

weighted query: $A_{0.5}, B_{0.7}, C_{1.0}$

form weighted vector:

(0.5, 0.7, 1.0)

retrieve lists of document ids from inverted file corresponding to A, B and C with weights

doc3 (.6)	doc2 (.6)	doc2 (.4)
doc4 (.7)	doc3 (.8)	doc4 (.7)
doc7 (.3)	doc5 (.9)	doc9 (.6)
doc8 (.5)	doc6 (.3)	doc11 (.3)
doc10 (.5)	doc8 (.5)	doc12 (.6)
	doc12 (.2)	

doc1: 0.85
doc2: 0.86

$\text{sim}((0.5, 0.7, 1.0), (0.0, 0.6, 0.4)) = 0.86$

Use of inverted file

- sort (rank) list according to similarity coefficient.
- retrieve first 'N' ranked objects.
- present ranked list to user.
- offer to retrieve next 'N'.
- Note that so far we have not retrieved any documents; this is particularly important if the ids are URLs - we don't need to start downloading web pages in order to rank them.

Use of inverted file

- proximity queries eg Q1: "A B" Q2: "A(3)B" (A...B)
 - postings file holds positional information
 - proceed as for 'A and B'
 - keep positional information in $(A \cap B)$ list
 - filter $(A \cap B)$ list:
 - for Q1 $\text{pos}(A) = \text{pos}(B) - 1$
 - for Q2 $|\text{pos}(B) - \text{pos}(A)| < 3$
- now we can distinguish 'Venetian blind' from 'blind Venetian'
- in principle this should help precision without affecting recall too much

Pros and cons of inverted file

- can be used for Boolean, weighted and positional queries
- query processing can be completed without accessing data file
- number of hits for single term is available from dictionary
- expensive to update if information objects change content.
- demanding storage requirements (dictionary+inverted file approx same size as original data)

Summary of key points: inverted file index

- standard relational databases do not provide suitable indexing for search on sets of index terms.
- standard SQL is not good at expressing 'search-engine' type queries
- inverted file structures are purpose made for these types of system
- storing frequencies/weights in the dictionary and inverted file allows for vector model queries
- storing positional information allows proximity queries, "Knowledge Management" v "Management Knowledge"

Efficiency is vital

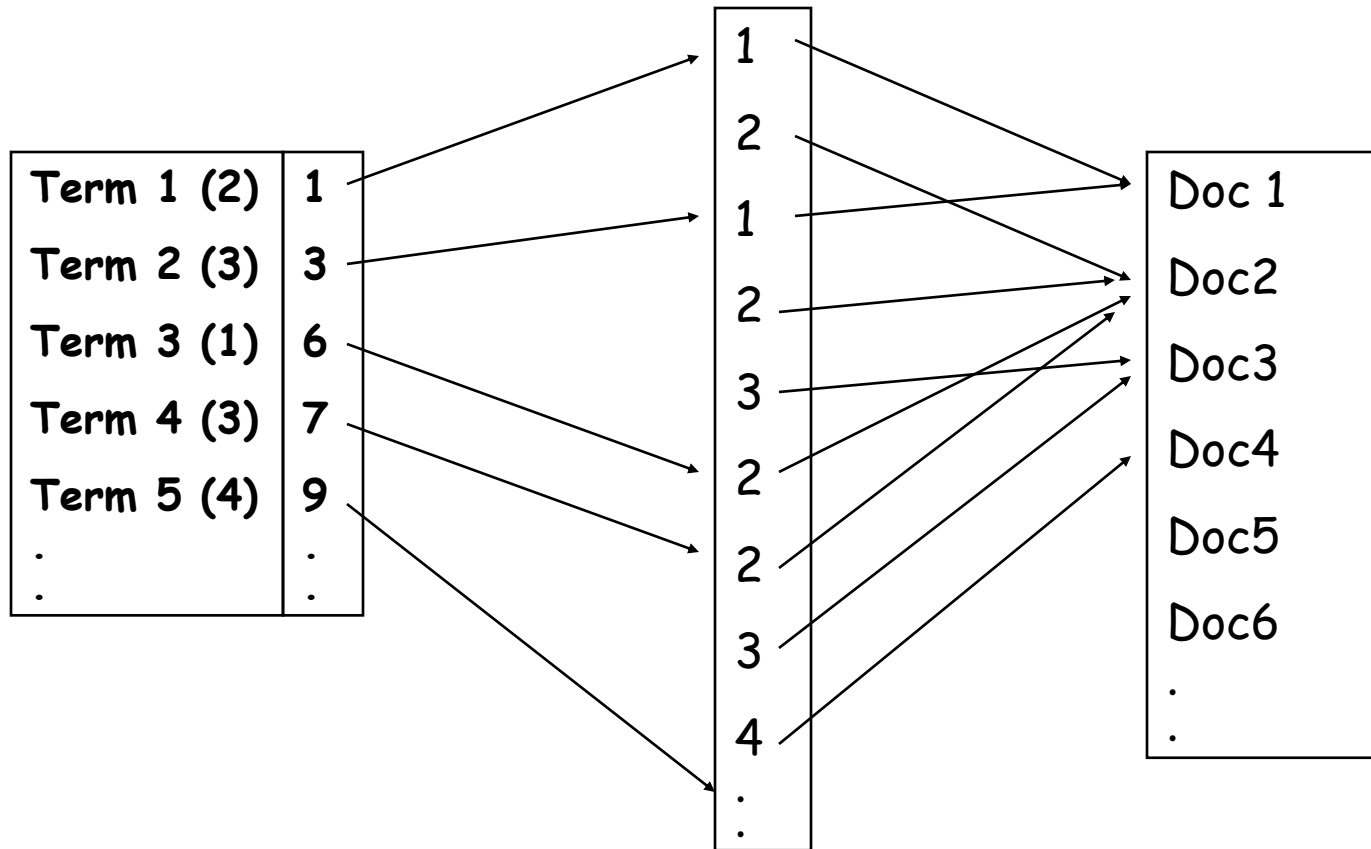
- Reminder: Google finds documents which match your keywords; this must be done EFFICIENTLY – cant just go through each document from start to end for each keyword
- So, cache stores copy of document, and also a “cut-down” version of the document for searching: just a “bag of words”, a sorted list (or array/vector/...) of words appearing in the document (with links back to full document)
- Try to match keywords against this list; if found, then return the full document
- Even cleverer: dictionary and inverted file...

Inverted file structure

dictionary

Inverted or postings file

Data file



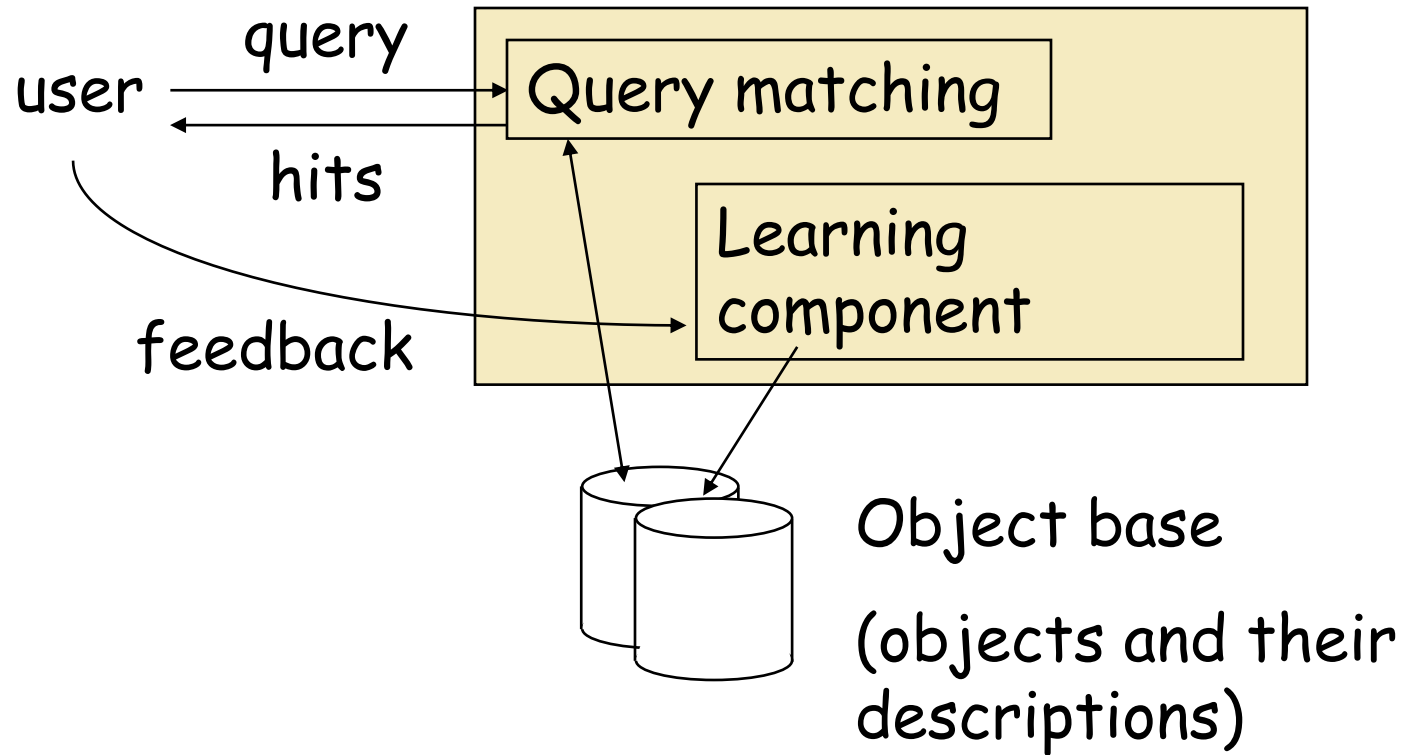
IR vs DBMS

	DBMS	IR
match	exact	partial or best match
inference	deduction	induction
model	deterministic	probabilistic
data	record/field	text document
query language	artificial	natural?
query	complete	incomplete
specification		
items wanted	matching	relevant
error response	sensitive	insensitive

Retrieval of documents

- IR was developed for bibliographic systems. We shall refer to 'documents', but the technique extends beyond items of text.
- central to IR is representation of a document by a set of 'descriptors' or 'index terms' ("words in the document").
- searching for a document is carried out (mainly) in the 'space' of index terms.
- we need a language for formulating queries, and a method for matching queries with document descriptors.

architecture



basic notation

Given a list of m documents, D , and a list of n index terms, T , we define $w_{i,j}$ to be a weight associated with the i^{th} keyword and the j^{th} document.

For the j^{th} document, we define an index term vector, $\underline{d}_j$:

$$\underline{d}_j = (w_{1,j}, w_{2,j}, \dots, w_{n,j})$$

For example: $D = \{d_1, d_2, d_3\}$,

$T = \{\text{pudding, jam, traffic, lane, treacle}\}$

$$\underline{d}_1 = (1, 1, 0, 0, 0),$$

$$\underline{d}_2 = (0, 0, 1, 1, 0),$$

$$\underline{d}_3 = (1, 1, 1, 1, 0)$$

set theoretic, Boolean model

- Queries are Boolean expressions formed using keywords, eg:
 $(\text{'Jam'} \vee \text{'Treacle'}) \wedge \text{'Pudding'} \wedge \neg \text{'Lane'} \wedge \neg \text{'Traffic'}$
- Query is re-expressed in disjunctive normal form (DNF)
CF: $T = \{\text{pudding, jam, traffic, lane, treacle}\}$

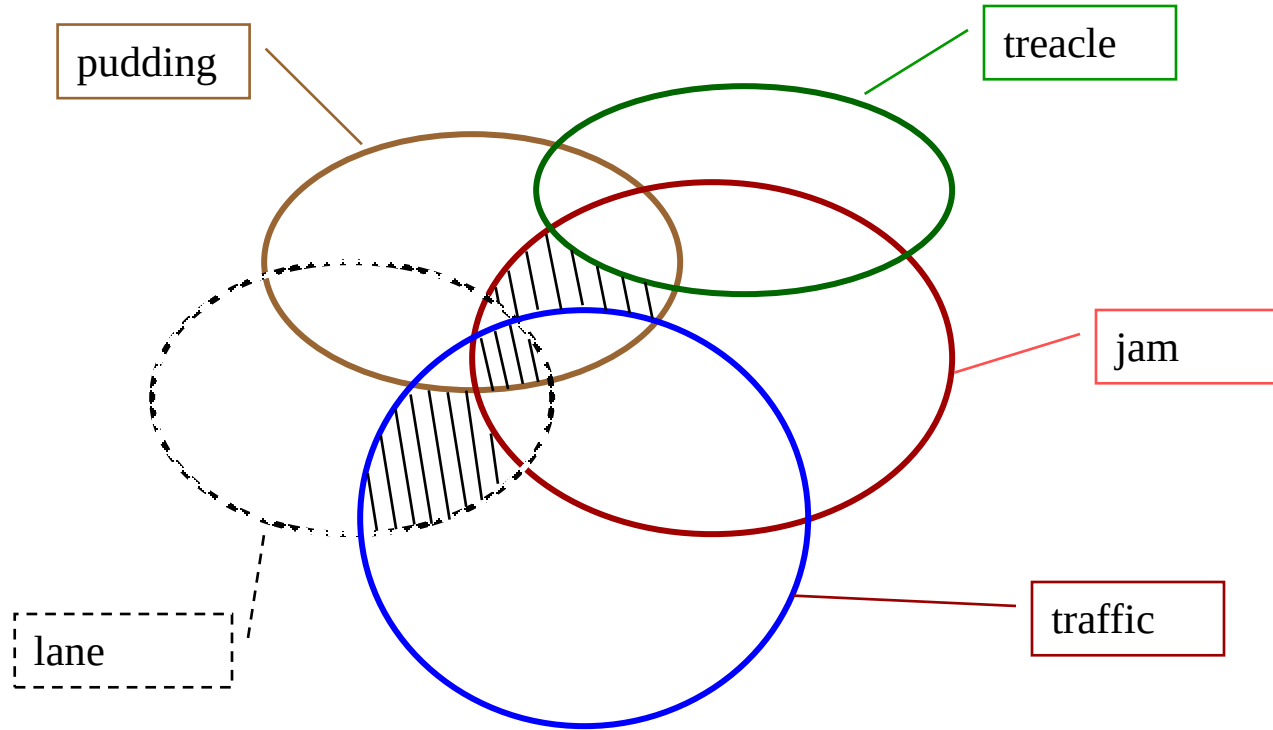
eg $(1, 1, 0, 0, 0) \vee (1, 0, 0, 0, 1) \vee (1, 1, 0, 0, 1)$

To match a document with a query:

$$\begin{aligned} \text{sim}(\underline{d}, q_{\text{DNF}}) &= 1 \text{ if } \underline{d} \text{ is equal to a component of } q_{\text{DNF}} \\ &= 0 \text{ otherwise} \end{aligned}$$

$$(1, 1, 0, 0, 0) \vee (1, 0, 0, 0, 1) \vee (1, 1, 0, 0, 1)$$

$T = \{\text{pudding, jam, traffic, lane, treacle}\}$



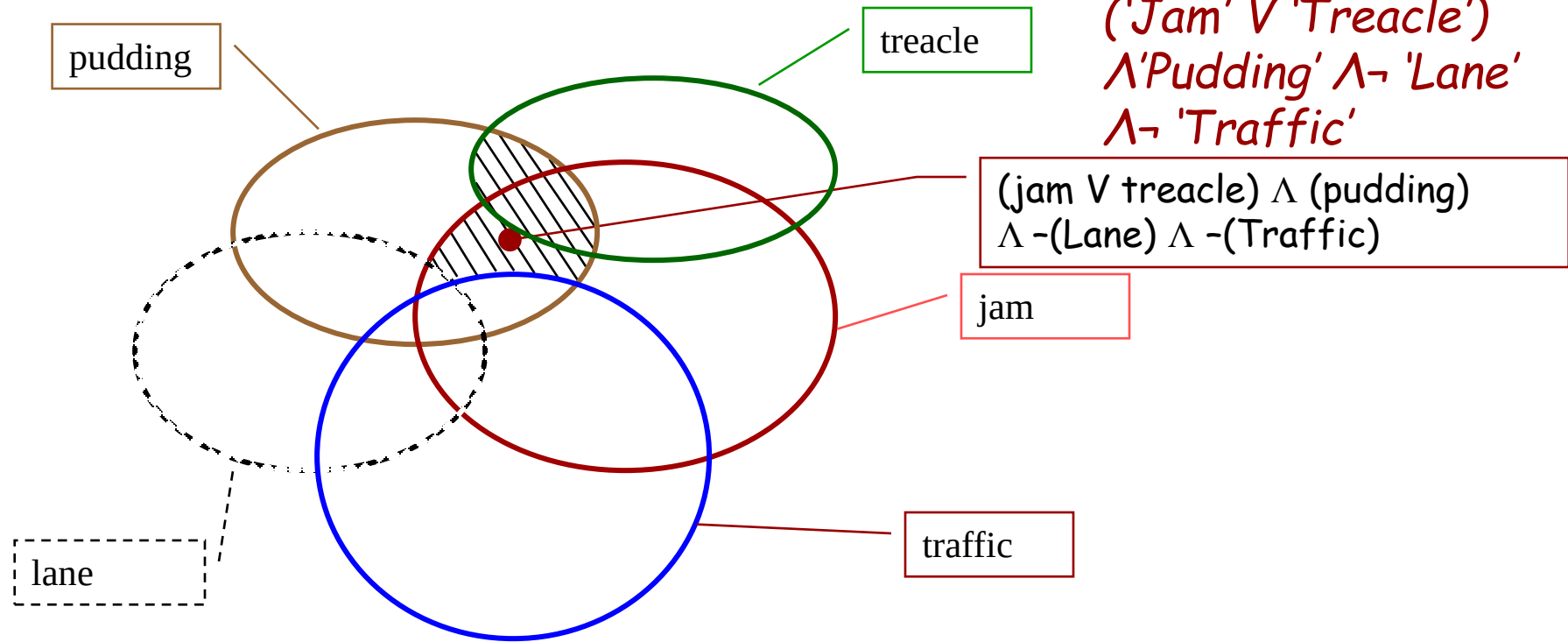
$$\underline{d}_1 = (1, 1, 0, 0, 0), \checkmark$$

$$\underline{d}_2 = (0, 0, 1, 1, 0), \times$$

$$\underline{d}_3 = (1, 1, 1, 1, 0), \times$$

collecting results

$T = \{\text{pudding, jam, traffic, lane, treacle}\}$



Query:

$(\text{'Jam'} \vee \text{'Treacle'})$
 $\wedge \text{'Pudding'} \wedge \neg \text{'Lane'}$
 $\wedge \neg \text{'Traffic'}$

Answer: $\underline{d}_1 = (1, 1, 0, 0, 0)$ Jam pud recipe

Statistical vector model

- weights, $-1 \leq w_{i,j} \leq 1$, no longer binary-valued
- query also represented by a vector

$$\underline{q} = (w_{1q}, w_{2q}, \dots, w_{nq})$$

$$\text{— eg } \underline{q} = (1.0, 0.6, 0.0, 0.0, 0.8)$$

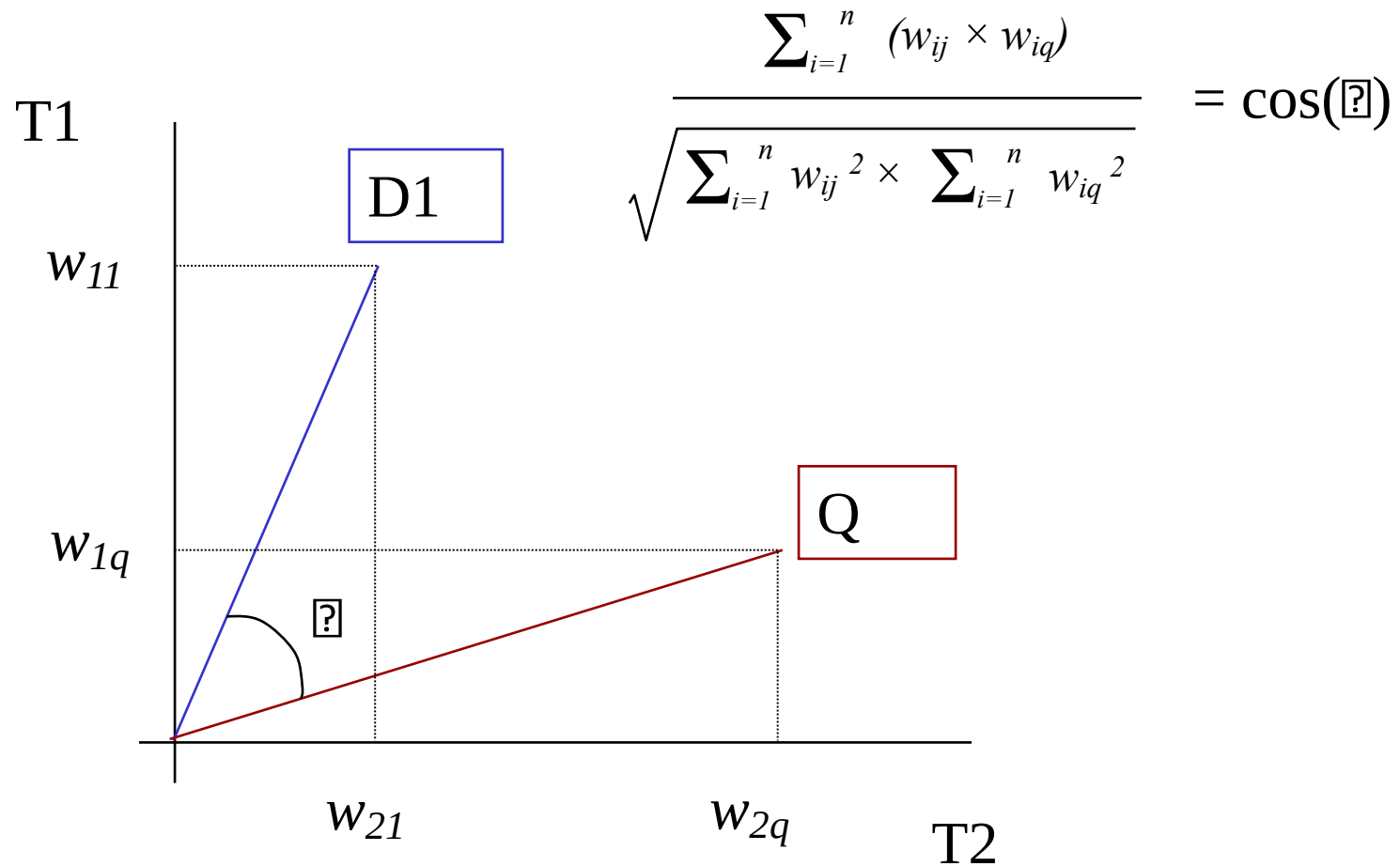
CF: $T = \{\text{pudding, jam, traffic, lane, treacle}\}$

to match j^{th} document with a query:

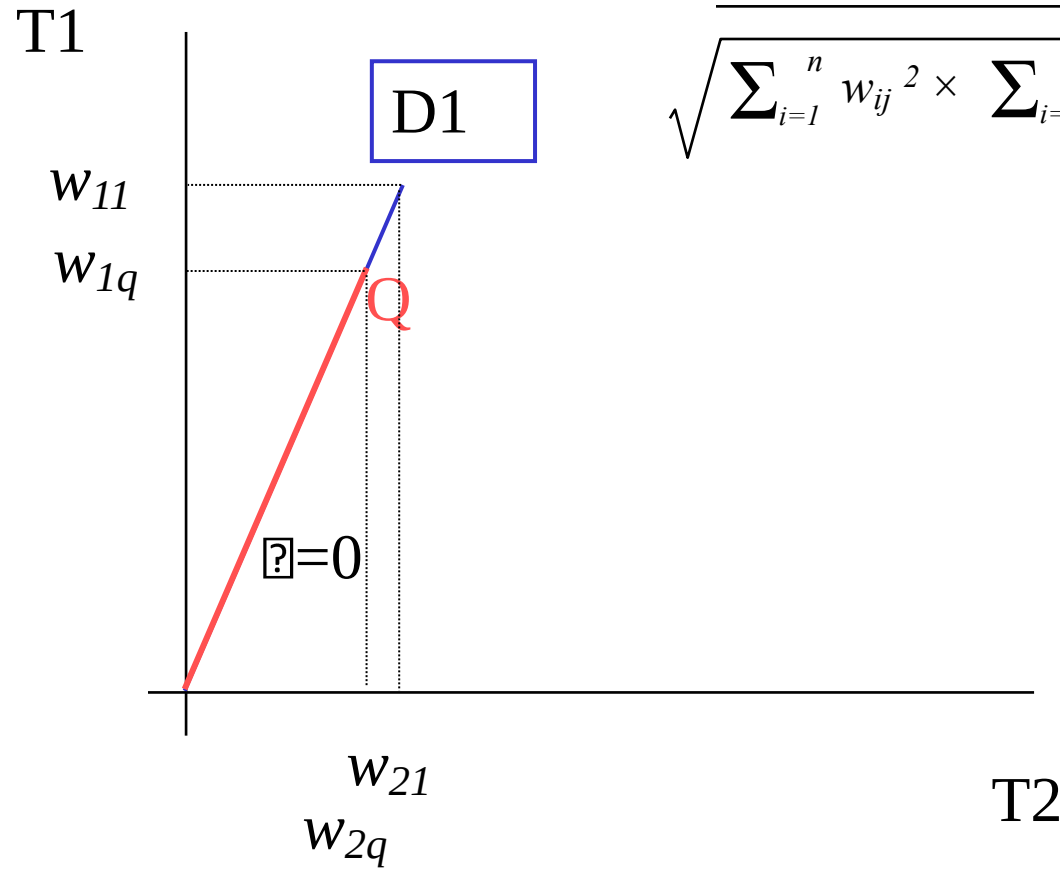
$$\text{sim}(\underline{d}_j, \underline{q}) =$$

$$\frac{\sum_{i=1}^n (w_{ij} \times w_{iq})}{\sqrt{\sum_{i=1}^n w_{ij}^2 \times \sum_{i=1}^n w_{iq}^2}}$$

Cosine coefficient

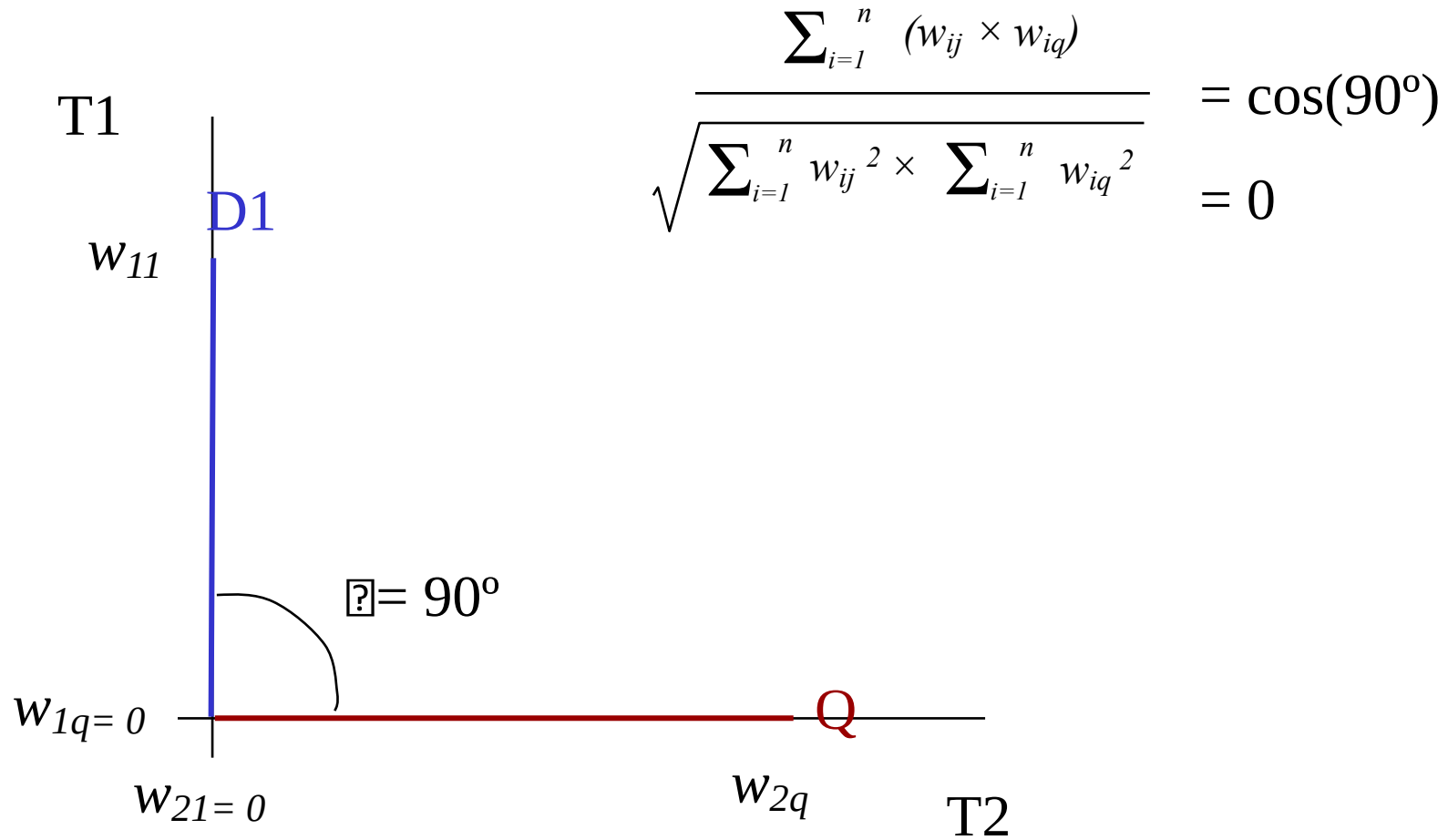


Cosine coefficient



$$\frac{\sum_{i=1}^n (w_{ij} \times w_{iq})}{\sqrt{\sum_{i=1}^n w_{ij}^2 \times \sum_{i=1}^n w_{iq}^2}} = \cos(0) = 1$$

Cosine coefficient



$$\underline{q} = (1.0, 0.6, 0.0, 0.0, 0.8)$$

$$\underline{d}_1 = (0.8, 0.8, 0.0, 0.0, 0.2) \quad \text{Jam pud recipe}$$

$$\sum_{i=1}^n (W_{ij} \times W_{iq})$$

$$= 0.8 \times 1.0 + 0.8 \times 0.6 + 0.0 \times 0.0 + 0.0 \times 0.0 + 0.2 \times 0.8$$

$$= 1.44$$

$$\sum_{i=1}^n W_{ij}^2 = 0.8^2 + 0.8^2 + 0.0^2 + 0.0^2 + 0.2^2 = 1.32$$

$$\sum_{i=1}^n W_{iq}^2 = 1.0^2 + 0.6^2 + 0.0^2 + 0.0^2 + 0.8^2 = 2.0$$

$$\frac{\sum_{i=1}^n (W_{ij} \times W_{iq})}{\sqrt{\sum_{i=1}^n W_{ij}^2 \times \sum_{i=1}^n W_{iq}^2}} = \frac{1.44}{\sqrt{1.32 \times 2.0}} = 0.89$$

$$\underline{q} = (1.0, 0.6, 0.0, 0.0, 0.8)$$

$$\underline{d}_2 = (0.0, 0.0, 0.9, 0.8, 0),$$

DoT Report

$$\sum_{i=1}^n (w_{ij} \times w_{iq})$$

$$= 0.0 \times 1.0 + 0.0 \times 0.6 + 0.9 \times 0.0 + 0.8 \times 0.0 + 0.0 \times 0.8$$

$$= 0.0$$

$$\sum_{i=1}^n w_{ij}^2 = 0.0^2 + 0.0^2 + 0.9^2 + 0.8^2 + 0.0^2 = 1.45$$

$$\sum_{i=1}^n w_{iq}^2 = 1.0^2 + 0.6^2 + 0.0^2 + 0.0^2 + 0.8^2 = 2.0$$

$$\frac{\sum_{i=1}^n (w_{ij} \times w_{iq})}{\sqrt{\sum_{i=1}^n w_{ij}^2 \times \sum_{i=1}^n w_{iq}^2}} = \frac{0.0}{\sqrt{1.45 \times 2.0}} = 0.0$$

$$\underline{q} = (1.0, 0.6, 0.0, 0.0, 0.8)$$

$$\underline{d}_3 = (0.6, 0.9, 1.0, 0.6, 0.0)$$

Radio Traffic Report

$$\sum_{i=1}^n (w_{ij} \times w_{iq})$$

$$= 0.6 \times 1.0 + 0.9 \times 0.6 + 1.0 \times 0.0 + 0.6 \times 0.0 + 0.0 \times 0.8$$

$$= 1.14$$

$$\sum_{i=1}^n w_{ij}^2 = 0.6^2 + 0.9^2 + 1.0^2 + 0.6^2 + 0.0^2 = 2.53$$

$$\sum_{i=1}^n w_{iq}^2 = 1.0^2 + 0.6^2 + 0.0^2 + 0.0^2 + 0.8^2 = 2.0$$

$$\frac{\sum_{i=1}^n (w_{ij} \times w_{iq})}{\sqrt{\sum_{i=1}^n w_{ij}^2 \times \sum_{i=1}^n w_{iq}^2}} = \frac{1.14}{\sqrt{2.53 \times 2.0}} = 0.51$$

collecting results

CF: $T = \{\text{pudding, jam, traffic, lane, treacle}\}$

$\underline{q} = (1.0, 0.6, 0.0, 0.0, 0.8)$

Rank	document vector	document	(sim)
1.	$\underline{d}_1 = (0.8, 0.8, 0.0, 0.0, 0.2)$	<i>Jam pud recipe</i>	(0.89)
2.	$\underline{d}_3 = (0.6, 0.9, 1.0, 0.6, 0.0)$	<i>Radio Traffic Report</i>	(0.51)

Discussion: Set theoretic model

- Boolean model is simple, queries have precise semantics, but it is an 'exact match' model, and does not Rank results
- Boolean model popular with bibliographic systems; available on some search engines
- Users find Boolean queries hard to formulate
- Attempts to use set theoretic model as basis for a partial-match system: Fuzzy set model and the extended Boolean model.

Discussion: Vector Model

- Vector model is simple, fast and results show leads to 'good' results.
- Partial matching leads to ranked output
- Popular model with search engines
- Underlying assumption of term independence (not realistic! Phrases, collocations, grammar)
- Generalised vector space model relaxes the assumption that index terms are pairwise orthogonal (but is more complicated).

questions raised

- Where do the index terms come from?
- (ALL the words in the source documents?)
- What determines the weights?
- How well can we expect these systems to work for practical applications?
- How can we improve them?
- How do we integrate IR into more traditional DB management?

Some issues to be resolved

- Synonyms
 - football / soccer, tap / faucet: search for one, find both?
- homonyms
 - lead (metal or leash?), tap: find both, only want one?
- local/global contexts determine “good” terms
 - football articles: won’t mention word ‘football’;
will have particular meaning for the word ‘goal’
- Precoordination (proximity query): multi-word terms
 - “Venetian blind” vs “blind Venetian”

Evaluation/Effectiveness measures

- **effort** - required by the users in formulation of queries
- **time** - between receipt of user query and production of list of 'hits'
- **presentation** - of the output
- **coverage** - of the collection
- **recall** - the fraction of relevant items retrieved
- **precision** - the fraction of retrieved items that are relevant
- **user satisfaction** – with the retrieved items

Better hits: Query Broadening

- User unaware of collection characteristics is likely to formulate a 'naïve' query
- query broadening aims to replace the initial query with a new one featuring one or other of:
 - new index terms
 - adjusted term weights
- One method uses feedback information from the user
- Another method uses a thesaurus / term-bank / ontology

Relevance Feedback

From response to initial query, gather relevance information

H = set of all hits

$H_R = R$ = set of retrieved, relevant hits

$H_{NR} = H - R$ = set of retrieved, non-relevant hits

replace query $\underline{q}$ with replacement query $\underline{q}'$:

$$\underline{q}' = \alpha \times \underline{q}$$

$$+ \beta \times \sum_{d_i \in H_R} \underline{d}_i / |H_R|$$

$$- \gamma \times \sum_{d_i \in H_{NR}} \underline{d}_i / |H_{NR}|$$

note: this moves the query vector closer to the centroid of the “relevant retrieved” document vectors and further from the centroid of the “non-relevant retrieved” documents.

Using terms from relevant documents

- We expect documents that are similar to one another in meaning (or usefulness) to have similar index terms.
- The system creates a replacement query (q') based on q , but adds index terms that have been used to index known relevant documents, increases the relative weight of index terms in q that are also found in relevant documents, and reduces the weight of terms found in non-relevant documents.

How does this help?

- It could help if documents were being missed because of the synonym problem. The user uses the word 'jam', but some recipes use 'jelly' instead. Once a hit that uses 'jelly' has been recognized as relevant, then 'jelly' will appear in the next version of the query. Now hits may use 'jelly' but not 'jam'.
- Conversely, it can help with the homonym problem. If the user wants references to 'lead' (the metal), and gets documents relating to dog-walking, then by marking the dog-walking references as not relevant, key words associated with dog-walking will be reduced in weight

pros and cons of feedback

- If γ is set = 0, ignore non-relevant hits, a positive feedback system; often preferred
- the feedback formula can be applied repeatedly, asking user for relevance information at each iteration
- relevance feedback is generally considered to be very effective for “high-use” systems
- one drawback is that it is not fully automatic.

Simple feedback example:

$T = \{\text{pudding, jam, traffic, lane, treacle}\}$

$\underline{d}_1 = (0.8, 0.8, 0.0, 0.0, 0.4),$

$\underline{d}_2 = (0.0, 0.0, 0.9, 0.8, 0.0),$

$\underline{d}_3 = (0.8, 0.0, 0.0, 0.0, 0.8)$

$\underline{d}_4 = (0.6, 0.9, 0.5, 0.6, 0.0)$

Recipe for jam pudding

DoT report on traffic lanes

Recipe for treacle pudding

Radio item on traffic jam in Pudding Lane

Display first 2 documents that match the following query:

$\underline{q} = (1.0, 0.6, 0.0, 0.0, 0.0)$

$\underline{r} = (0.91, 0.0, 0.6, 0.73)$

Retrieved documents are:

$\underline{d}_1$: Recipe for jam pudding **relevant**

$\underline{d}_4$: Radio item on traffic jam in Pudding Lane **not relevant**

Positive and Negative Feedback

Suppose we set α and β to 0.5, γ to 0.2

$$\underline{q}' = \alpha \times \underline{q} + \beta \times \sum_{\underline{d}_i \in H_R} \underline{d}_i / |H_R| - \gamma \times \sum_{\underline{d}_i \in H_{NR}} \underline{d}_i / |H_{NR}|$$

$$= 0.5 \underline{q} + 0.5 \underline{d}_1 - 0.2 \underline{d}_4$$

$$\begin{aligned} = & 0.5 \times (1.0, 0.6, 0.0, 0.0, 0.0) \\ & + 0.5 \times (0.8, 0.8, 0.0, 0.0, 0.4) \\ & - 0.2 \times (0.6, 0.9, 0.5, 0.6, 0.0) \end{aligned}$$

$$= (0.78, 0.52, -0.1, -0.12, 0.2)$$

(Note $|H_n| = 1$ and $|H_{nr}| = 1$)

Simple feedback example:

$T = \{\text{pudding, jam, traffic, lane, treacle}\}$

$\underline{d}_1 = (0.8, 0.8, 0.0, 0.0, 0.4),$

$\underline{d}_2 = (0.0, 0.0, 0.9, 0.8, 0.0),$

$\underline{d}_3 = (0.8, 0.0, 0.0, 0.0, 0.8)$

$\underline{d}_4 = (0.6, 0.9, 0.5, 0.6, 0.0)$

Display first 2 documents that match the following query:

$\underline{q}' = (0.78, 0.52, -0.1, -0.12, 0.2)$

$\underline{r}' = (0.96, 0.0, 0.86, 0.63)$

Retrieved documents are:

$\underline{d}_1$: Recipe **relevant** jam

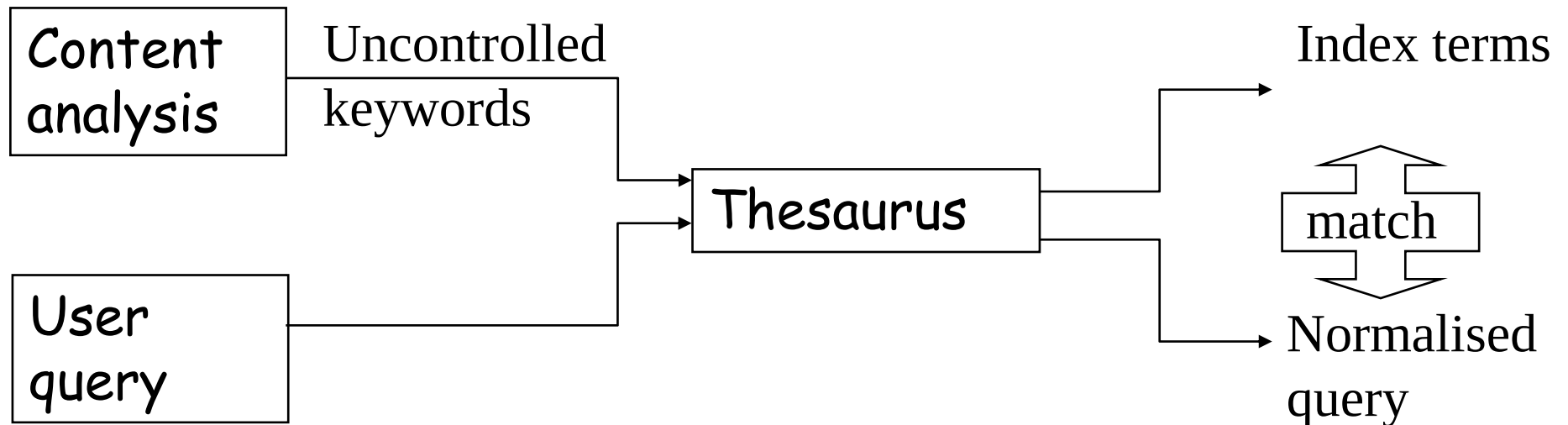
$\underline{d}_3$: Recipe **relevant** treacle

Thesaurus

- a thesaurus or ontology may contain
 - controlled vocabulary of terms or phrases describing a specific restricted topic,
 - synonym classes,
 - hierarchy defining broader terms (hypernyms) and narrower terms (hyponyms)
 - classes of ‘related’ terms.
- a thesaurus or ontology may be:
 - generic (as Roget’s thesaurus, or WordNet)
 - specific to a certain domain of knowledge, eg medical

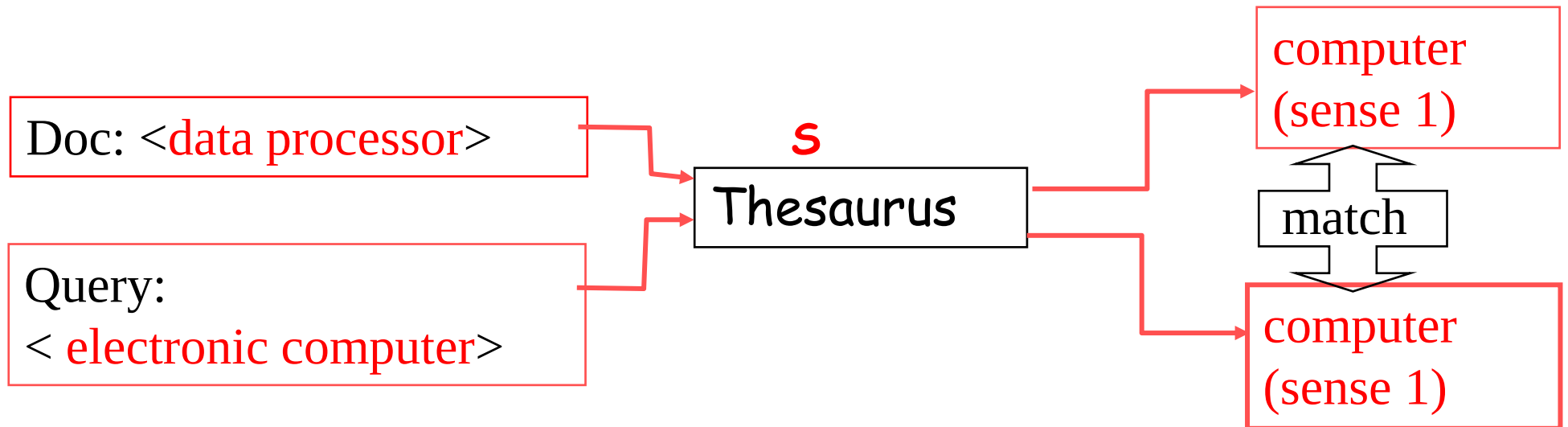
Language normalisation

by replacing words from documents and query words with synonyms from a controlled language, we can improve precision and recall:



Thesaurus use

- replace term in document and/or query with term in controlled language
- replace term in query with related or broader term to increase recall
- suggest to user narrower terms to increase precision



Questions to think about

- Why is traditional database unsuited to retrieval of unstructured information?
- How would you re-express a Boolean query, eg (A or B or (C and not D)), in disjunctive normal form?
- For the matching coefficient, $\text{sim}(\cdot, \cdot)$ show that $0 \leq \text{sim}(\cdot, \cdot) \leq 1$, and that $\text{sim}(\underline{a}, \underline{a}) = 1$.
- Compare and contrast the 'vector' and 'set theoretic' models in terms of power of representation of documents and queries.

Information Retrieval (Google search)

IR v Database SQL

Inverted file for efficient match

Boolean set theoretic model v weighted vector model

Worked examples

Evaluation

Query broadening to improve matching